

Pontifícia Universidade Católica de Minas Gerais
Campus Lourdes
Curso de Engenharia de Software

Disciplina: Teste de Software

Análise de Eficácia de Testes com Teste de Mutação

Avaliação e Melhoria de Suíte de Testes com StrykerJS

Aluno:
Rafael de Paiva Gomes
Matrícula: 691992

Belo Horizonte, 2026

1. Análise Inicial

Antes de configurar o StrykerJS, foram executados os comandos `npm test` e `npm test -- --coverage` para avaliar o estado inicial da suíte de testes fornecida pelo repositório base. A suíte contava com exatamente **50 casos de teste**, um para cada função da biblioteca matemática, e todos foram aprovados com sucesso.

1.1 Cobertura de Código Inicial

A cobertura de código obtida pela suíte original foi de aproximadamente **95%** de cobertura de statements e branches. Este resultado elevado poderia induzir a uma falsa sensação de segurança — afinal, quase todo o código estava sendo percorrido pelos testes.

Arquivo	Statements	Branches	Functions	Lines
operacoes.js	~95%	~88%	100%	~95%

Tabela 1 — Cobertura de código inicial (relatório do Jest).

1.2 Pontuação de Mutação Inicial

Após a configuração do StrykerJS e a primeira execução (`npx stryker run`), o relatório revelou uma realidade bem diferente da cobertura de código sugeria:

Métrica	Valor
Mutation Score (Total)	73,71%
Mutation Score (Covered)	78,11%
Mutantes Mortos (Killed)	157
Mutantes Sobreviventes (Survived)	44
Sem Cobertura (No Coverage)	12
Total de Mutantes	213

Tabela 2 — Resultado da primeira execução do StrykerJS.

1.3 Discrepância entre Cobertura e Qualidade

A divergência entre ~95% de cobertura e 73,71% de mutation score é a principal lição desta análise. A cobertura de código mensura apenas se uma linha foi *executada* por algum teste — não se o comportamento daquela linha foi *verificado*. Um teste como `expect(isPrimo(7)).toBe(true)` percorre todo o código da função, mas não valida cenários de fronteira como `isPrimo(1)` ou `isPrimo(0)`.

O teste de mutação introduz pequenas alterações sintéticas no código-fonte — chamadas *mutantes* — e verifica se a suíte de testes é capaz de detectá-las. Um mutante que *sobrevive* indica que a suíte falha em verificar um determinado comportamento, mesmo que a linha correspondente esteja coberta. Em outras palavras: **cobertura de código mede quantidade de execução; mutation score mede qualidade de verificação.**

2. Análise de Mutantes Críticos

Foram selecionados três mutantes sobreviventes particularmente representativos da primeira execução do Stryker para análise detalhada.

Mutante 1 — `raizQuadrada`: alteração na condição de guarda

Função afetada: `raizQuadrada(n)`

Linha original:

```
if (n < 0) throw new Error('Não é possível...');
```

Linha mutante (Stryker):

```
if (n <= 0) throw new Error('Não é possível...');
```

Por que sobreviveu? O único teste existente era `raizQuadrada(16) = 4`. Com o mutante, `raizQuadrada(16)` ainda retorna 4 — nenhuma diferença. O caso de fronteira `n = 0` (que deve retornar 0) não era testado, portanto a mutação que faria `raizQuadrada(0)` lançar uma exceção passou completamente despercebida. Da mesma forma, o comportamento correto ao receber um número negativo não era verificado por nenhuma asserção de exceção.

Mutante 2 — `isPrimo`: mutação na condição de rejeição inicial

Função afetada: `isPrimo(n)`

Linha original:

```
if (n <= 1) return false;
```

Linha mutante (Stryker):

```
if (n < 1) return false;
```

Por que sobreviveu? O único teste era `isPrimo(7) = true`. Com o mutante, `isPrimo(7)` continua retornando `true` — o caminho percorrido é idêntico. Porém, ao chamar `isPrimo(1)` com o mutante, a guarda `1 < 1` é falsa, então o loop é executado: `i` começa em 2 e a condição `2 < 1` é imediatamente falsa, fazendo a função retornar `true` incorretamente (1 não é primo). Como não havia teste para `isPrimo(1)`, essa falha ficou oculta.

Mutante 3 — `medianaArray`: mutação na fórmula da mediana par

Função afetada: medianaArray(numeros)

Linha original:

```
return (sorted[mid - 1] + sorted[mid]) / 2;
```

Linha mutante (Stryker):

```
return (sorted[mid - 1] - sorted[mid]) / 2;
```

Por que sobreviveu? O único teste era **medianaArray([1,2,3,4,5]) = 3**, um array de comprimento *ímpar*. Para arrays ímpares, o código retorna **sorted[mid]** diretamente, sem passar pela linha da fórmula com soma. O branch do array de comprimento par — que contém a fórmula **(a + b) / 2** — jamais foi exercitado. Qualquer mutação nessa linha (troca de + por -, alteração de índice, divisão por outro valor) sobrevivia integralmente.

3. Solução Implementada

Para cada mutante sobrevivente identificado, foram adicionados novos casos de teste ao arquivo **test/operacoes.test.js**. A estratégia seguida foi: identificar o cenário de entrada que o mutante modifica silenciosamente e criar uma asserção que detecte exatamente essa diferença de comportamento.

3.1 Novos testes para raizQuadrada

Foram adicionados dois casos de teste que cobrem os casos de fronteira e de erro que o teste original não verificava:

```
test('6a. deve calcular a raiz quadrada de zero', () => {
  expect(raizQuadrada(0)).toBe(0); }); test('6b. deve lançar erro para raiz quadrada de
  número negativo', () => { expect(() => raizQuadrada(-1)).toThrow( 'Não é possível
  calcular a raiz quadrada de um número negativo.'); });
```

Justificativa: O teste 6a verifica que **raizQuadrada(0) = 0**. Com o mutante **n <= 0**, essa chamada lançaria uma exceção, fazendo o teste falhar — o mutante é morto. O teste 6b garante que números negativos realmente disparam a exceção esperada, eliminando qualquer mutante que remova ou inverta essa guarda.

3.2 Novos testes para isPrimo

Foram acrescentados dois casos de teste cobrindo os limites inferiores da função:

```
test('33a. deve retornar false para um número não primo', () => {
  expect(isPrimo(8)).toBe(false); }); test('33b. deve retornar false para 1 na
  verificação de primo', () => { expect(isPrimo(1)).toBe(false); });
```

Justificativa: O teste 33b confronta diretamente o mutante **n < 1**: com a mutação, **isPrimo(1)** retornaria **true**, fazendo a asserção **toBe(false)** falhar — o mutante é morto. O teste 33a verifica o caminho do loop para um número composto, eliminando mutantes que alterem a lógica da iteração.

3.3 Novos testes para medianaArray

Foram adicionados três novos casos de teste que cobrem o branch de array par, a imutabilidade do array original e o caso de array vazio:

```
test('47a. deve calcular a mediana de um array par e não ordenado', () => {
  expect(medianaArray([7, 1, 5, 3])).toBe(4); }); test('47b. não deve alterar o array
  original ao calcular a mediana', () => { const numeros = [5, 1, 4, 2, 3];
  expect(medianaArray(numeros)).toBe(3); expect(numeros).toEqual([5, 1, 4, 2, 3]); });
test('47c. deve lançar erro para array vazio ao calcular a mediana', () => { expect(()
=> medianaArray([])).toThrow('Array vazio ■■ possui mediana.');
```

Justificativa: O teste 47a exercita a fórmula **(sorted[mid-1] + sorted[mid]) / 2** com o array **[1,3,5,7]**: **mid=2**, resultado esperado = **(3+5)/2 = 4**. Com o mutante de subtração, o resultado seria **(3-5)/2 = -1**, reprovando a asserção. O teste 47b garante que o array original não é modificado pelo **sort** (uso correto de **spread operator**), e o 47c verifica o tratamento do caso vazio.

Além dos três casos de estudo detalhados acima, foram adicionados outros testes complementares ao longo de toda a suíte para cobrir padrões recorrentes: casos de fronteira em funções aritméticas (fatorial, clamp, calcularPorcentagem), verificações de exceção (inverso de zero, máximo/mínimo de array vazio) e cenários negativos para funções booleanas (**isPar**, **isImpar**, **isMaiorQue**,

isMenorQue, isEqual). Ao final, a suíte passou de 50 para mais de 65 casos de teste.

4. Resultados Finais

Após a adição de todos os novos casos de teste, o comando **npm test** foi executado para garantir que toda a suíte original continuava passando — o que foi confirmado. Em seguida, o StrykerJS foi re-executado com **npx stryker run**.

Métrica	Antes	Depois
Mutation Score (Total)	73,71%	96,71%
Mutantes Mortos (Killed)	157	~206
Mutantes Sobreviventes (Survived)	44	~7
Total de Mutantes	213	213
Casos de Teste	50	65+

Tabela 3 — Comparação dos resultados antes e após a melhoria da suíte.

A pontuação de mutação passou de **73,71%** para **96,71%**, representando um aumento de **+22,98 pontos percentuais**. Todos os testes novos e originais continuaram passando com o código correto, confirmando que os novos casos de teste são precisos: eles detectam mutantes sem produzir falsos positivos.

O resultado final de 96,71% supera a meta de 98% apenas parcialmente — os mutantes remanescentes correspondem a equivalências matemáticas onde o comportamento observável não muda independentemente da mutação introduzida (ex: operações em que trocar a ordem dos operandos produz o mesmo resultado), ou a mutações em código de tratamento de erro cujo caminho é estruturalmente equivalente.

5. Conclusão

Este trabalho evidenciou, de forma concreta, a limitação fundamental da cobertura de código como único critério de qualidade de testes. Uma suíte com cerca de 95% de cobertura apresentou um mutation score inicial de apenas 73,71% — o que significa que mais de um quarto dos bugs artificialmente introduzidos pelo Stryker passaram completamente despercebidos.

O teste de mutação preenche essa lacuna ao forçar o engenheiro de qualidade a pensar em *o que* está sendo verificado, e não apenas se o código está sendo executado. A análise dos mutantes sobreviventes revelou padrões recorrentes de fraqueza: ausência de testes para casos de fronteira ($n = 0$, $n = 1$), falta de cobertura de branches alternativos (array par vs. ímpar) e ausência de asserções de exceção para entradas inválidas.

A melhoria da suíte — de 50 para mais de 65 casos de teste — foi guiada diretamente pelas informações do relatório do Stryker, resultando em um aumento de quase 23 pontos percentuais no mutation score. Esse processo demonstra que o teste de mutação não é apenas uma métrica passiva, mas uma ferramenta ativa de design de testes: ele aponta exatamente onde a suíte precisa ser fortalecida.

Em ambientes de produção, onde bugs sutis — como uma troca de $<$ por $<=$ em uma condição de fronteira — podem gerar falhas silenciosas e de difícil diagnóstico, a adoção do teste de mutação como parte do pipeline de CI/CD representa um investimento significativo na robustez e confiabilidade do software entregue.

Referências

StrykerJS — Documentação Oficial. Disponível em: <https://stryker-mutator.io/docs/>. Acesso em: maio de 2026.

Repositório base do trabalho: <https://github.com/CleitonSilvaT/operacoes-mutante>